МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра анализа данных и искусственного интеллекта

Допустить к защите
заведующий кафедрой
д-р тех. наук, доцент
_____ А. В. Коваленко
«___» _____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА (БАКАЛАВРСКАЯ РАБОТА)

РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ-ЕЖЕДНЕВНИКА С ИСПОЛЬЗОВАНИЕМ МЕТОДА ГЕЙМИФИКАЦИИ

Работу выполнил(а) _____ И. Д. Ерюшев
Направление подготовки 02.03.03 Математическое обеспечение и администрирование информационных систем
Направленность (профиль) Технология программирования

Научный руководитель
канд. пед. наук, доцент _____ В. А. Акиньшина

Нормоконтролер
канд. физ.-мат. наук, доцент _____ Г. В. Калайдина

Краснодар 2026

РЕФЕРАТ

Выпускная квалификационная работа содержит 64 с., 5 рис., 8 табл., 20 источников.

ГЕЙМИФИКАЦИЯ, UNITY, C#, NEXT.JS, TYPESCRIPT, FIREBASE, FIRESTORE, МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, ВЕБ-ПРИЛОЖЕНИЕ, РОДИТЕЛЬСКИЙ КОНТРОЛЬ, ЭКСПЕРТНАЯ СИСТЕМА, АНАЛИТИКА, РЕКОМЕНДАЦИИ.

Объектом исследования является процесс разработки клиент-серверной программной системы, предназначенной для мотивации ребенка к выполнению реальных повседневных задач средствами геймификации.

Предмет исследования составляют архитектурные, программные и интерфейсные решения, обеспечивающие взаимодействие игрового Unity-клиента ребенка, родительского веб-приложения и облачного хранилища данных.

Цель работы заключается в развитии прототипа мобильного приложения-ежедневника до целостной системы с синхронизацией заданий, адаптивным игровым интерфейсом, аналитикой выполнения и объяснимыми рекомендациями для родителя.

В работе применены методы объектно-ориентированного проектирования, сравнительного анализа, прототипирования пользовательских интерфейсов, событийного моделирования и логического экспертного вывода.

Результатом является программный комплекс, включающий Unity-клиент ребенка, родительскую веб-панель, общую модель данных Firestore, жизненный цикл задания, систему игровых наград, аналитические показатели и экспертный модуль рекомендаций. Геолокационные квесты обоснованы как направление дальнейшего развития.

СОДЕРЖАНИЕ

Реферат 2

Содержание 3

Введение 6

1 Анализ исходного прототипа и архитектурное развитие системы 9

1.1 Общая характеристика исходной версии 9

1.2 Новая постановка задачи 10

1.3 Требования к улучшенной версии 11

1.4 Критерии успешности модернизации 12

1.5 Общая архитектура 14

1.6 Unity-клиент ребенка 15

1.7 Родительское веб-приложение 16

1.8 Модель данных Firestore 17

1.9 Cloud Functions и безопасность бизнес-логики 20

1.10 Визуальная система и адаптивный интерфейс 21

2 Реализация улучшений и результаты 22

2.1 Переработка экрана входа 22

2.2 Карта мира как игровое пространство 23

2.3 Доска приключений и система заданий 23

2.4 Синхронизация заданий между Unity и Web 24

2.5 Система наград и игровой прогресс 25

2.6 Импорт и организация графических ресурсов 26

2.7 Адаптивность интерфейса 27

2.8 Исправление проблем интеграции и тестирования 28

2.9 Изменение качества продукта по сравнению с первой версией .. 29
2.10 Проверяемые пользовательские сценарии 31
2.11 Техническая проверка 31
2.12 Ограничения текущей версии 32
2.13 Итоговые результаты 33
3 Расширение системы: геолокационные квесты, аналитика и экспертные рекомендации 34
3.1 Обоснование направлений дальнейшей модернизации 34
3.2 Расширение модели реального квеста 35
3.3 Общий жизненный цикл расширенного задания 36
3.4 Связь планируемых функций с текущей архитектурой 37
3.5 Назначение и пользовательский сценарий геолокационного задания 39
3.6 Выбор технического подхода и SDK 40
3.7 Реализация активной проверки посещения в Unity 41
3.8 Реализация автоматического геозонирования на Android 42
3.9 Модель данных Firestore для геолокационных квестов 43
3.10 Безопасность, приватность и ограничения геолокационного модуля 45
3.11 Проверка работоспособности геолокационных сценариев 46
3.12 Реализованный аналитический модуль и задачи родителя 47
3.13 Событийная модель и собираемые показатели 47
3.14 Агрегация данных и ежедневные витрины статистики 48
3.15 Оценка интереса ребенка к различным типам квестов 50
3.16 Представление аналитики в родительском веб-интерфейсе 51

3.17 Постановка задачи рекомендации 51
3.18 Сведения и признаки, используемые рекомендательной системой 52
3.19 Создание логической экспертной системы 53
3.20 Механизм работы экспертной системы в приложении 55
3.21 Модель на основе дерева решений 56
3.22 Гибридный подход к рекомендациям 57
3.23 Объяснимость рекомендаций для родителя 57
3.24 Интеграция экспертного модуля в Firebase и веб-панель 58
3.25 Этапы разработки аналитико-рекомендательного модуля 58
3.26 Технические риски и способы их снижения 59
3.27 Результаты реализации и направления развития 60
Заключение 61
Список использованных источников 63

ВВЕДЕНИЕ

Развитие цифровых технологий изменило не только способы обучения и развлечения детей, но и характер ежедневного взаимодействия внутри семьи. Ребенок все чаще воспринимает смартфон или компьютер как естественную среду отдыха, общения и получения впечатлений. Родитель, напротив, часто рассматривает цифровое устройство как источник отвлечения от учебы, домашних обязанностей и полезных привычек. Возникает противоречие: полностью исключить цифровую среду из жизни ребенка невозможно, но ее можно направить в сторону воспитательных и развивающих задач.

Одним из способов решения этой проблемы является геймификация. Игровые механики позволяют придать повседневным действиям понятную цель, обратную связь и эмоциональное подкрепление. Если уборка комнаты, чтение, помощь по дому или выполнение учебного задания представлены не только как требование взрослого, но и как часть приключения, ребенку проще увидеть в них смысл. Особенно важным становится то, что цифровая среда не заменяет реальное действие, а связывает его с игровым прогрессом. [1–3]

Исходная версия проекта представляла собой Unity-приложение, в котором ребенок мог перемещаться по карте, участвовать в боях, получать опыт и золото, а также видеть задания. Такой прототип показывал работоспособность самой идеи: реальные дела могут быть встроены в RPG-структуру и восприниматься как квесты. Однако по мере развития проекта стало ясно, что одного игрового клиента недостаточно. Для практического семейного сценария необходим второй участник системы – родитель, который формулирует задания, назначает награды, проверяет выполнение и наблюдает за прогрессом.

Поэтому следующая стадия разработки была связана не только с улучшением внешнего вида, но и с переходом от отдельного игрового прототипа к распределенной системе. В проект добавлено родительское веб-приложение на Next.js и TypeScript, использующее Firebase Authentication и Firestore. Unity-клиент получил REST-интерфейс доступа к Firebase, модели профиля ребенка, игрового состояния и реальных заданий. В результате ребенок взаимодействует с игрой, а родитель – с веб-панелью; обе части используют общую модель данных.

Актуальность работы определяется необходимостью создания цифровых инструментов, которые не усиливают разрыв между ребенком и родителем, а помогают выстраивать понятное взаимодействие. Если сравнивать проект с обычными списками задач, проект связывает реальные обязанности с игровой мотивацией. Если сравнивать его с обычной развлекательной игрой, он не замыкается на экране, а поощряет действия во внешнем мире. По сравнению с жесткими системами контроля, он сохраняет элемент добровольности, выбора и эмоционального вовлечения.

Целью этой работы является разработка и описание улучшенной версии геймифицированного ежедневника, в которой исходный Unity-прототип был преобразован в более целостную программную систему с веб-интерфейсом родителя, синхронизацией заданий, адаптивным интерфейсом и расширенной визуальной составляющей.

Для достижения цели были поставлены следующие задачи:

– проанализировать ограничения исходной версии приложения и определить направления развития;

– спроектировать общую архитектуру системы, включающую Unity-клиент ребенка, веб-приложение родителя и облачное хранилище данных;

– реализовать родительскую веб-панель для создания детей, настройки Unity-входа и управления заданиями;

– настроить синхронизацию заданий и игрового состояния через Firebase Authentication и Firestore, а также подготовить Cloud Functions для доверенных серверных операций;

– доработать Unity-клиент: авторизацию ребенка, загрузку профиля, отображение заданий, отправку выполненных квестов на проверку и получение наград;

– провести визуальную переработку интерфейса: адаптивные экраны, карту с объектными локациями, стилизованную доску приключений и подключение графических ресурсов;

– выполнить проверку работоспособности основных сценариев и описать полученные результаты;

– спроектировать направления дальнейшего развития системы: геолокационные задания, расширенную аналитику выполнения квестов и рекомендательный модуль;

– реализовать экспертную рекомендательную систему: определить события и аналитические признаки, сформировать правила вывода, рассчитать оценку рекомендации и предусмотреть дальнейшее развитие на основе дерева решений. [20]

Практическая значимость работы связана с тем, что проект перестает быть изолированным игровым прототипом и становится основой для реального семейного приложения. Родитель получает инструмент постановки задач, ребенок – игровую оболочку для выполнения этих задач, а общая база данных обеспечивает согласованность действий между двумя интерфейсами.

1 Анализ исходного прототипа и архитектурное развитие системы

1.1 Общая характеристика исходной версии

Первая версия проекта была ориентирована преимущественно на проверку игровой гипотезы. В Unity были реализованы основные элементы RPG-clicker adventure: карта мира, несколько локаций, переход к боевой сцене, враги, характеристики игрока, награды и базовая система заданий. Такой подход быстро показал, может ли механика кликера и RPG-прогресса быть использована как оболочка для реальных задач ребенка. [4]

Сильной стороной исходного прототипа была простая и понятная структура игрового цикла. Ребенок видел карту, выбирал доступную локацию, вступал в бой, наносил урон врагу, получал опыт и золото. Параллельно существовала идея реальных квестов, за которые также можно получать игровые награды. Уже на этом этапе была заложена связь между внешней активностью и игровым развитием персонажа.

При этом первоначальная версия имела ограничения. Во-первых, задания были представлены скорее как локальный список, а не как полноценный семейный процесс. Отсутствовал удобный интерфейс родителя для создания квестов и проверки выполнения. Во-вторых, Unity-интерфейс оставался функциональным, но недостаточно выразительным: карта воспринималась как набор кнопок, а не как игровое пространство. В-третьих, требовалась адаптация к мобильным экранам: элементы, корректно выглядевшие на ПК, плохо масштабировались на телефоне.

Кроме того, в первой версии визуальная составляющая выполняла вспомогательную роль. Для детского продукта этого недостаточно, поскольку восприятие игры в значительной степени определяется целостностью мира: фонами, персонажами, врагами, предметами, иконками, панелями и состояниями интерфейса. Без стилистического единства даже

рабочая механика воспринимается как технический прототип, а не как законченный игровой опыт.

В итоге исходный проект выполнил свою задачу как MVP игровой части, но следующая стадия требовала развития в сторону реального использования: появления родительского интерфейса, синхронизации данных, улучшения пользовательского сценария и визуальной целостности.

1.2 Новая постановка задачи

После анализа исходного состояния проекта была сформулирована новая постановка задачи. Требовалось не просто добавить отдельные функции, а изменить уровень зрелости системы. Если первая версия отвечала на вопрос, можно ли встроить реальные задания в игру, то вторая версия должна была ответить на вопрос, как организовать полный цикл взаимодействия ребенка и родителя.

В новой версии система рассматривается как два связанных приложения. Первое приложение – Unity-клиент ребенка. Оно должно оставаться игровым: карта, локации, бои, награды, прогресс, визуальная атмосфера приключения. Второе приложение – веб-панель родителя. Оно должно быть более утилитарным: регистрация, создание семьи, добавление ребенка, выдача Unity-логина, создание заданий, просмотр выполненных заданий, одобрение или отклонение результата.

Между этими частями необходима общая облачная прослойка. В проекте эту роль выполняет Firebase. Authentication используется для разделения пользователей на родителя и ребенка. Firestore хранит профили, детей, задания и игровое состояние. Cloud Functions предполагаются для операций, которые в production-версии должны выполняться на серверной стороне: одобрение, отклонение и выдача награды. [10]

С точки зрения пользовательского сценария новая версия должна поддерживать следующую последовательность. Родитель входит в веб-приложение, создает семейный профиль и профиль ребенка, генерирует

сведения для Unity-входа, создает задание и назначает награду. Ребенок входит в Unity-приложение под своим аккаунтом, видит задание на доске приключений, выполняет его в реальной жизни, отправляет комментарий на проверку. Родитель видит отправленный квест в веб-панели и принимает решение. После одобрения ребенок получает внутриигровую награду.

Именно такой сценарий делает проект не просто игрой и не просто ежедневником, а связанной мотивационной системой. В ней действия одного пользователя имеют последствия в интерфейсе другого пользователя, а реальные действия ребенка получают цифровое подкрепление.

1.3 Требования к улучшенной версии

После анализа ограничений были выделены функциональные и нефункциональные требования к улучшенной версии проекта.

К функциональным требованиям относятся: регистрация и вход родителя в веб-приложение; создание семьи и профиля ребенка; хранение Unity-логина ребенка; создание заданий с названием, описанием, категорией и наградами; отображение списка заданий ребенка; фильтрация заданий по статусам; отправка задания ребенком на проверку; одобрение, отклонение и получение награды; хранение игрового состояния; поддержка dev-режима для локального тестирования без обязательной облачной конфигурации.

К требованиям Unity-клиента относятся: загрузка профиля ребенка после входа; построение карты доступных локаций; переход между картой, городом и боем; отображение реальных заданий на доске приключений; визуальное различение статусов задания; возможность отправить задание на проверку; возможность получить награду после одобрения; отображение уровня, опыта и золота; сохранение игрового прогресса.

К требованиям веб-приложения относятся: понятная структура родительской панели; создание и просмотр детей; отдельный раздел с данными Unity-входа; экран создания квеста; экран списка квестов; экран

проверки отправленных квестов; базовая аналитика; корректная работа с Firestore rules; возможность локального запуска для тестирования.

К нефункциональным требованиям относятся: адаптивность интерфейса Unity-клиента под разные пропорции экранов; сохранение читаемости текста на мобильных устройствах; визуальное единство графики; разделение ответственности между сервисами; расширяемость модели данных; возможность дальнейшего перехода от MVP-операций к серверной business logic через Cloud Functions. [6]

Отдельным требованием стала эстетическая переработка. В детском игровом приложении визуальная среда не является второстепенной. Она напрямую влияет на мотивацию и желание возвращаться в приложение. Поэтому карта должна выглядеть как игровая карта с объектными локациями, задания – как карточки на доске приключений, а фоновые изображения – корректно заполнять экран без искажений.

1.4 Критерии успешности модернизации

Для оценки результата были определены критерии успешности. Первым критерием является завершенность пользовательского цикла. Система должна позволять пройти путь от создания задания родителем до получения ребенком награды в игре. Даже если часть production-логики временно реализована прямыми запросами к Firestore, архитектура должна предусматривать перенос критических операций в Cloud Functions.

Вторым критерием является согласованность данных. Unity-клиент и веб-приложение должны использовать совместимую модель статусов, категорий, наград и игрового состояния. Наличие разных интерфейсов не должно приводить к расхождению данных.

Третьим критерием является улучшение пользовательского опыта. Карта должна восприниматься как часть игрового мира, а не как техническое меню. Доска заданий должна быть визуально связана с тематикой

приключения. Экран входа должен выглядеть как начало игры, а не как служебная форма.

Четвертым критерием является техническая устойчивость. Проект должен собираться без ошибок, а базовые сценарии должны быть проверяемы как в облачном режиме, так и в локальном dev-режиме. Это важно, поскольку разработка ведется итерационно, и на ранней стадии необходимо быстро тестировать отдельные элементы.

Пятым критерием является расширяемость. В дальнейшем проект должен позволять добавлять новые локации, врагов, предметы, категории заданий, аналитику, события, сезонные активности и более сложные правила награждения без полной переработки архитектуры.

1.5 Общая архитектура

Обновленный проект построен как распределенная система из трех основных частей: Unity-клиента ребенка, веб-приложения родителя и облачной инфраструктуры Firebase. Такое разделение соответствует реальному сценарию использования: ребенок взаимодействует с игровым миром, родитель управляет заданиями через браузер, а облачный слой обеспечивает хранение и синхронизацию данных.

Unity-клиент отвечает за игровую часть. В нем реализованы сцены входа, карты, боя и города. Клиент отображает игровые характеристики, доступные локации, врагов, прогресс и реальные квесты. В отличие от обычного ежедневника, Unity-клиент не показывает задания как сухой список. Они встроены в игровую метафору: доска приключений, награды, статусы, возможность получить золото и опыт.

Веб-приложение родителя отвечает за управленческую часть. Оно реализовано на Next.js и TypeScript. Родитель может авторизоваться, создать семью, добавить ребенка, увидеть данные для входа в Unity, создать квест, просмотреть активные задания и проверить отправленные ребенком результаты. Визуально веб-часть должна быть более спокойной и рабочей, поскольку ее основная задача – быстрое управление, а не игровое погружение. [13]

Firebase выполняет роль общей backend-платформы. Firebase Authentication используется для создания аккаунтов и выдачи токенов доступа. Firestore хранит документы пользователей, семей, детей, заданий, игрового состояния и аналитики. Cloud Functions используются как направление развития для операций, которые требуют доверенной серверной логики, например начисление награды после одобрения задания. [9]

Обмен между Unity и Firebase реализован через REST API. Это решение было выбрано потому, что оно позволяет управлять запросами напрямую из Unity, не привязывая проект к тяжелой SDK-интеграции на

ранней стадии. Unity-клиент отправляет HTTP-запросы, получает JSON-ответы, преобразует их в C#-модели и обновляет игровое состояние.

Обобщенная схема взаимодействия Unity-клиента, веб-клиента и сервисов Firebase представлена на рисунке 1.1.

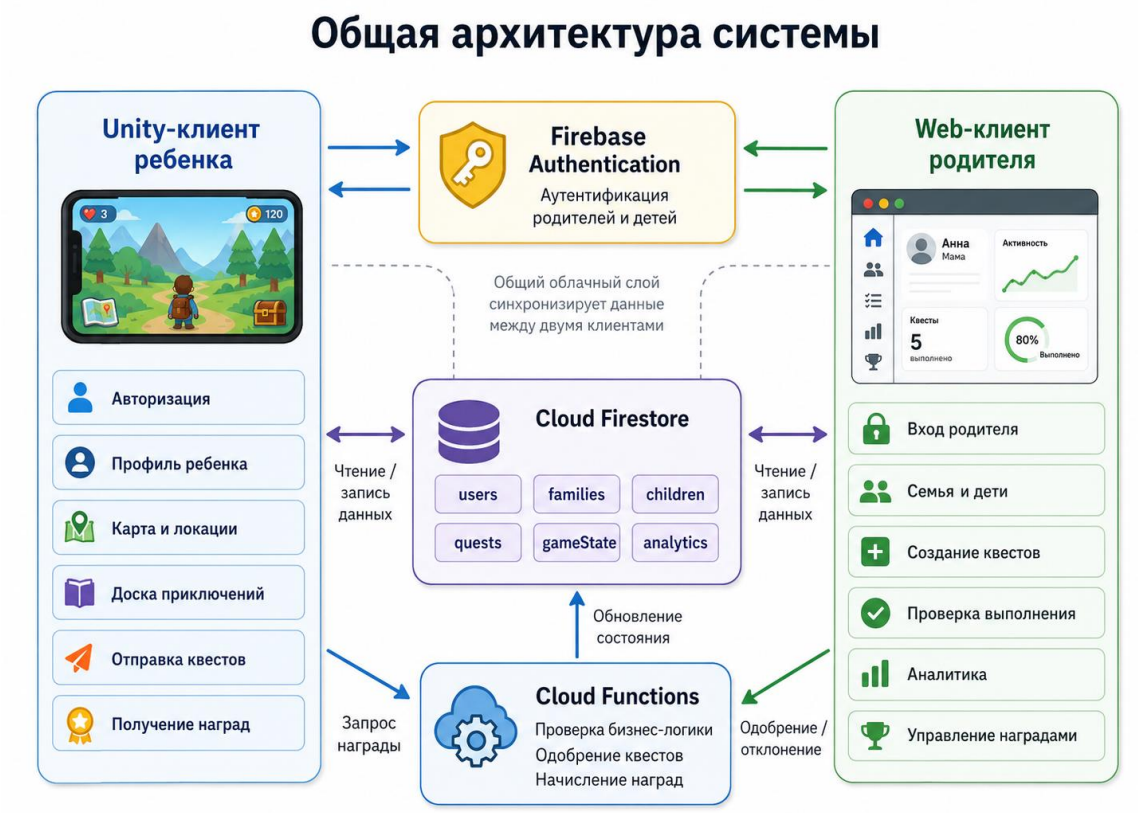


Рисунок 1.1 – Общая архитектура системы

1.6 Unity-клиент ребенка

Unity-клиент организован вокруг набора сервисов, которые инициализируются при запуске приложения. Центральным элементом является ServiceRegistry. Он создает и хранит ссылки на AuthService, PlayerProfileService, GameStateService, QuestService, RewardService, MapService, BattleService, AnalyticsService и другие вспомогательные классы. Такой подход упрощает доступ к общим сервисам из разных экранов и делает структуру проекта понятной. [5] [8]

Модель данных Unity-клиента включает несколько ключевых классов. UserProfile описывает текущего пользователя и содержит роль, семейный идентификатор, идентификатор ребенка, имя и email. ChildProfile хранит отображаемое имя, уровень, опыт и золото. GameState описывает игровые параметры: уровень, опыт, золото, здоровье, урон, шанс критического удара, открытые локации, пройденные подземелья и текущую точку карты. RealLifeQuest описывает реальное задание, его статус, категорию, награды и комментарии.

Игровая навигация разделена на сцены. LoginScreen отвечает за вход ребенка и загрузку профиля. MapScreen показывает карту мира и доступные локации. BattleScreen реализует боевой цикл. TownScreen содержит доску приключений и работу с заданиями. Такое разделение позволяет развивать каждую область независимо и не смешивать логику авторизации, карты, боя и заданий в одном классе.

Важным элементом стало сохранение dev-режима. Если Firebase-конфигурация отсутствует или разработчик тестирует интерфейс локально, приложение может создать демонстрационный профиль и показать тестовые задания. Это ускоряет разработку, поскольку не каждая визуальная или интерфейсная правка требует подключения к облачной базе.

При этом dev-режим не заменяет реальную архитектуру. Если Firebase настроен, Unity использует REST-клиент, выполняет вход ребенка через Identity Toolkit, загружает профиль из Firestore, получает задания и сохраняет прогресс. В итоге один и тот же клиент может работать как в локальном режиме разработки, так и в режиме интеграционного тестирования.

1.7 Родительское веб-приложение

Родительская часть проекта реализована как веб-приложение на Next.js с использованием React, TypeScript и Firebase Web SDK. Такой выбор обусловлен несколькими причинами. Во-первых, родителю удобнее управлять заданиями через браузер: можно быстро вводить длинные

описания, просматривать списки и проверять результаты. Во-вторых, веб-приложение проще распространять и обновлять, чем отдельное мобильное приложение для родителя. В-третьих, Next.js позволяет удобно разделять страницы, компоненты и сервисы. [13] [14] [15]

В структуре веб-приложения выделены страницы панели управления, настройки семьи, просмотра детей, создания заданий, списка квестов, проверки отправленных заданий и аналитики. Вынесены отдельные сервисы для работы с пользователями, семьями, детьми, квестами, Unity-логином и аналитикой. Компоненты интерфейса за счет этого меньше зависят от конкретных Firebase-запросов.

Типизация на TypeScript играет важную роль. В файле типов описаны UserProfile, Family, ChildProfile, Quest, DailyAnalytics и GameState. Эти интерфейсы соответствуют этим, которые используются в Firestore и Unity. Благодаря этому веб-приложение меньше зависит от неявных строковых структур и проще поддерживается при изменении модели.

Отдельно была добавлена поддержка Unity-входа ребенка. Родитель может создать сведения, которые затем используются ребенком в Unity-клиенте. В профиле ребенка предусмотрены поля unityLoginEmail, unityLoginPasswordForTesting и unityAuthUid. Для MVP это позволяет быстро проверить связку между веб-приложением и Unity без сложной процедуры приглашений.

Веб-приложение также реализует создание и управление квестами. Родитель задает название, описание, категорию, количество опыта и золота. После создания квест получает статус assigned. Когда ребенок отправляет выполнение на проверку, статус меняется на submitted. Родитель может одобрить или отклонить результат. После одобрения ребенок получает право забрать награду в Unity.

1.8 Модель данных Firestore

Firestore используется как единое хранилище состояния проекта. Основными коллекциями являются users, families и children. Документ users хранит данные аккаунта и роль пользователя. Документ families описывает семейную группу. Документ children хранит профиль ребенка, игровой прогресс и связанные подколлекции.

Внутри children/{childId} расположены подколлекции и документы, описывающие состояние конкретного ребенка. Документ gameState/main хранит уровень, опыт, золото, боевые параметры, открытые локации, текущую локацию и серию активности. Подколлекция quests хранит реальные задания. Подколлекция rewardLedger предназначена для учета наград. Подколлекция analyticsDaily может использоваться для ежедневной статистики.

Структура хранения данных в Cloud Firestore показана на рисунке 1.2.

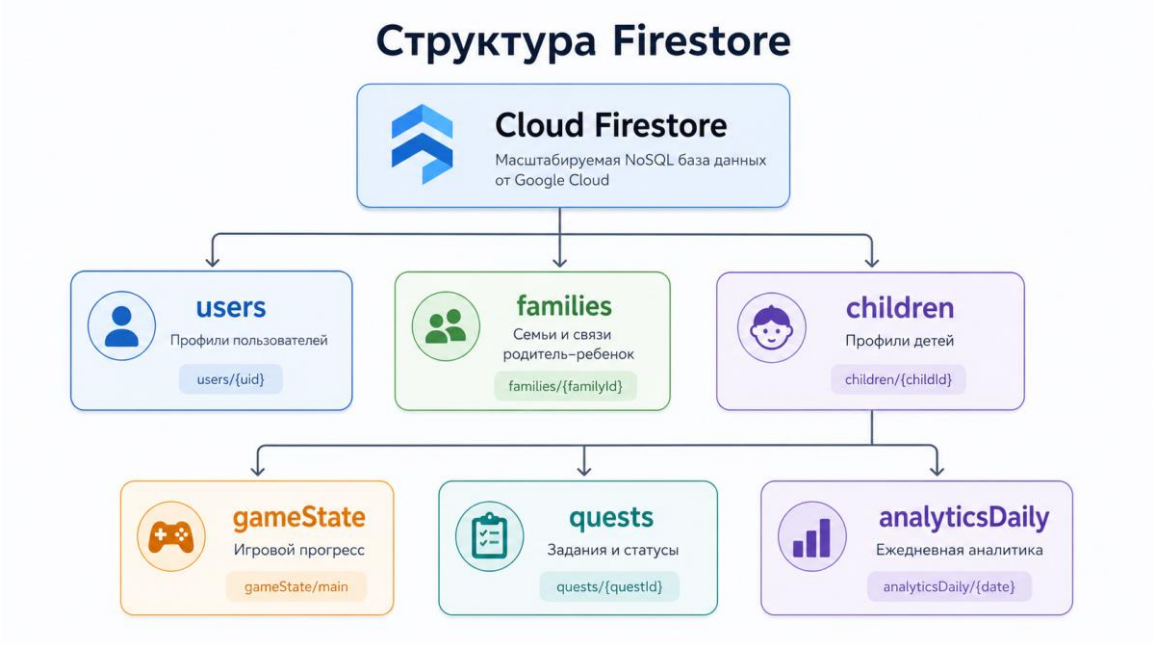


Рисунок 1.2 – Структура Firestore

Модель задания включает название, описание, категорию, статус, награду в опыте и золоте, создателя, даты изменения статуса, комментарий ребенка, комментарий родителя и ссылку на доказательство выполнения. Такая структура позволяет развивать систему без изменения основного жизненного цикла. Например, в будущем можно добавить фотоотчет,

повторяемые задания, дедлайны, сложность, предметные награды или цепочки квестов.

Статусы задания образуют конечный набор состояний: assigned, submitted, approved, rejected, claimed и cancelled. Такая модель удобна для синхронизации между Unity и веб-приложением. Unity показывает ребенку доступные действия на основании статуса. Веб-приложение показывает родителю, какие задания требуют проверки. Firestore rules ограничивают действия пользователей в зависимости от роли.

Жизненный цикл задания и переходы между основными статусами представлены на рисунке 1.3.

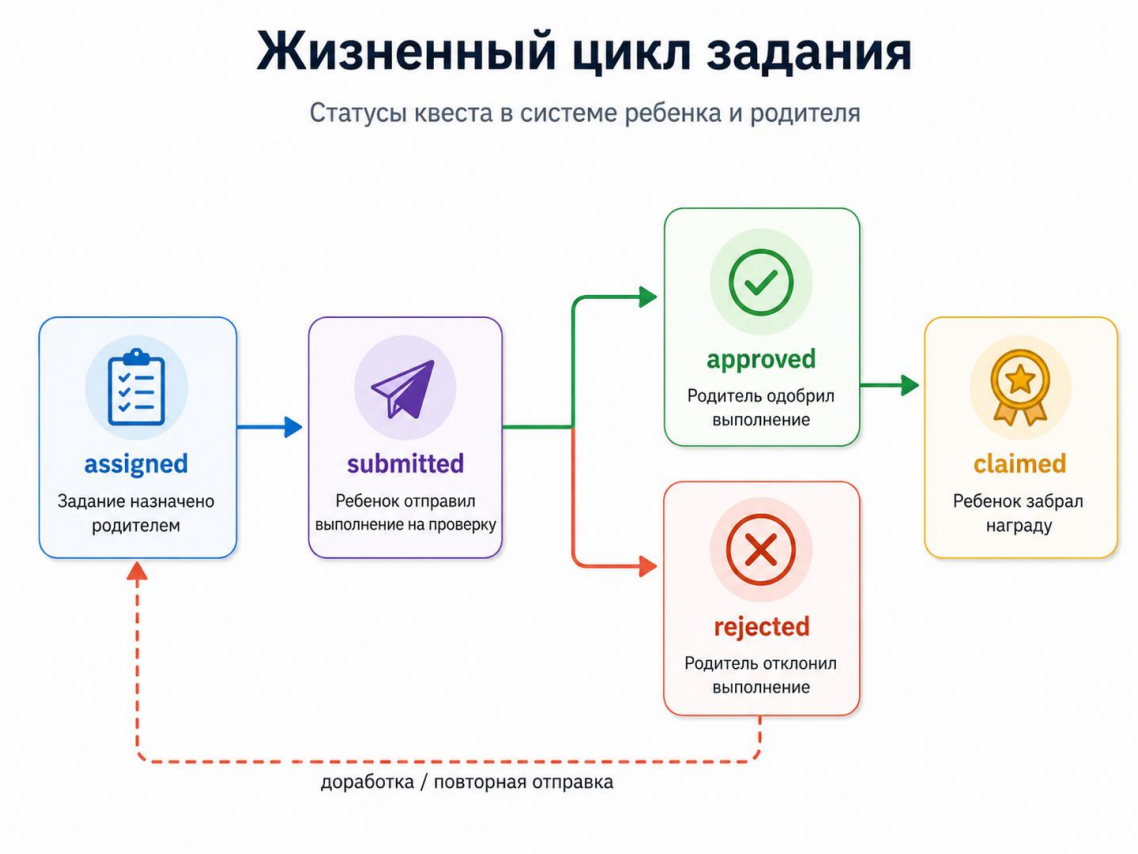


Рисунок 1.3 – Жизненный цикл задания

Важен и другой аспект: разделение прав доступа. Родитель может управлять данными детей своей семьи. Ребенок может читать свои задания и игровое состояние, отправлять назначенное или отклоненное задание на проверку и забирать одобренную награду. В production-версии начисление

наград должно выполняться через Cloud Functions, чтобы клиент не мог самостоятельно изменить критические сведения.

1.9 Cloud Functions и безопасность бизнес-логики

Cloud Functions рассматриваются как слой доверенной серверной логики. В MVP-версии часть операций может выполняться прямым обновлением документов Firestore, чтобы ускорить проверку интерфейса. Однако для итоговой архитектуры важно вынести операции одобрения, отклонения и выдачи награды на серверную сторону.

Причина заключается в том, что игровые награды имеют значение для прогресса. Если клиент сможет произвольно начислять опыт и золото, система потеряет надежность. Серверная функция должна проверять роль пользователя, статус задания, принадлежность ребенка к семье и допустимость перехода статуса. Только после этого она может изменить документ задания и обновить gameState или rewardLedger.

В Unity-клиенте предусмотрен вызов claimQuestReward через Cloud Functions. Если функция недоступна вовремя MVP-тестирования, реализован fallback-сценарий прямого обновления Firestore. Это компромисс между удобством разработки и будущей безопасностью. Такой подход позволяет уже сейчас тестировать пользовательский путь, но сохраняет направление для production-реализации.

Для веб-приложения аналогичный подход применим к approveQuest и rejectQuest. В ранней версии родительская панель может напрямую обновлять статус квеста, поскольку Firestore rules ограничивают доступ ролью родителя. В дальнейшем эти операции также целесообразно перенести в Cloud Functions, чтобы все ключевые переходы статусов проходили через единый слой проверки.

В итоге архитектура проекта учитывает текущую стадию разработки и будущие требования безопасности. MVP остается достаточно простым для

тестирования, но модель данных и сервисная структура не мешают перейти к более строгой серверной логике.

1.10 Визуальная система и адаптивный интерфейс

Одним из главных направлений улучшения стала визуальная переработка Unity-клиента. В проект была добавлена папка Graph с графическими материалами, после чего ресурсы были нарезаны и разложены по категориям: фоны, враги, герой, иконки, предметы, карта, панели, портреты, квесты и городские элементы. На основе этих изображений были созданы игровые и UI-префабы. [16]

Для обращения к графике в коде используется ArtLibrary. Этот класс предоставляет методы Background, Hero, Enemy, Item, Icon, Panel, Quest и Map. Вместо прямой загрузки ресурсов из разных мест экраны получают спрайты через единую библиотеку. Это упрощает замену графики и поддерживает единый стиль.

Адаптивность реализована на уровне Runtime UI. Был добавлен SafeAreaFitter, который учитывает safe area экрана и помогает корректно размещать интерфейс на мобильных устройствах с разными пропорциями и вырезами. Для фонов реализован AspectFillImage: изображение масштабируется так, чтобы заполнить экран без искажения пропорций. Это важно для стартового экрана и карты, где фон задает общее впечатление.

CanvasScaler настроен на масштабирование относительно портретной базовой резолюции. Так можно интерфейсу лучше выглядеть на телефоне, не превращая элементы в слишком маленькие или слишком растянутые. Отдельные панели, кнопки, карточки и изображения получают стабильные размеры и позиции, чтобы пользовательский интерфейс оставался читаемым.

В результате Unity-клиент стал ближе к полноценному мобильному приложению. Стартовый экран воспринимается как вход в приключение, карта – как игровое пространство, а задания – как элементы мира, а не как временный технический список.

2 Реализация улучшений и результаты

2.1 Переработка экрана входа

Экран входа является первой точкой контакта ребенка с приложением. В исходной версии он выполнял преимущественно техническую функцию: ввод логина и пароля, dev-вход и переход к карте. В новой версии экран был переработан с учетом игрового контекста.

Фон экрана заменен на приключенческое изображение, которое масштабируется через aspect-fill. Благодаря этому на разных экранах сохраняется ощущение цельной сцены. Основная форма входа размещена в стилизованной карточке, выполненной в виде панели или пергамента. Вместо абстрактного декоративного элемента используется портрет героя, связывающий авторизацию с игровой идентичностью ребенка.

Поля ввода и кнопки были размещены внутри safe area. Это решает проблему, при которой на телефоне элементы могли располагаться слишком близко к краям или перекрываться системными областями. Текстовые статусы были приведены к понятным русскоязычным формулировкам: загрузка профиля, вход, dev-вход, ошибки авторизации.

Важным улучшением стала поддержка ПК-тестирования. Поскольку разработка и проверка часто выполняются в Unity Editor на компьютере, интерфейс должен позволять вводить логин и пароль мышью и клавиатурой. Для этого проект использует Input System UI Input Module, совместимый с активным новым Input System. Это устраняет конфликт со старым UnityEngine.Input и позволяет тестировать формы без мобильной сборки. [7]

В итоге экран входа перестал быть только служебным окном и стал частью игрового опыта. Он также стал технически надежнее за счет корректной работы с Input System и адаптивным размещением.

2.2 Карта мира как игровое пространство

Карта мира была одной из областей, где различие между прототипом и полноценным продуктом проявлялось особенно заметно. В первоначальной версии локации воспринимались как кнопки или строки с иконками. Такой подход удобен для быстрого тестирования, но он снижает ощущение приключения. Ребенок видит меню, а не карту.

В новой версии MapScreen был переработан. Фон карты загружается как отдельный спрайт и заполняет экран без искажений. Поверх него размещается прозрачная поверхность карты, на которой располагаются игровые объекты локаций. Каждая локация имеет собственную иконку: деревня, солнечный лес, кристальная пещера или маркер закрытой области. Кликабельным становится сам объект, а не прямоугольная кнопка рядом с ним.

Такое изменение важно не только эстетически. Оно меняет способ восприятия навигации. Когда локация выглядит как объект на карте, ребенок понимает ее как часть мира. Переход в бой или город становится не нажатием меню, а выбором точки путешествия.

Для каждой точки карты учитывается состояние открытия. Если локация доступна, она отображается активным объектом, а при нажатии выполняется переход к соответствующей сцене. Если локация закрыта, она отображается приглушенно и сопровождается информацией о требуемом уровне. Так можно заранее показывать ребенку цель развития: новые области мира существуют, но для доступа к ним нужно получить опыт.

Верхняя панель карты показывает имя героя, уровень, опыт и золото. Нижняя навигация содержит быстрые переходы к гильдии, инвентарю и настройкам. Даже если часть разделов пока находится в MVP-состоянии, сама структура интерфейса уже задает направление развития продукта.

2.3 Доска приключений и система заданий

Одной из ключевых доработок стала стилизация доски заданий. В исходном прототипе задания отображались функционально, но визуально не были достаточно связаны с игровым миром. В новой версии TownScreen был переработан как гильдейский экран, где реальные задания представлены как приключенческие поручения.

Фон города или гильдии создает ощущение места, куда герой приходит за заданиями. Центральная область оформлена как доска приключений. Внутри нее размещается прокручиваемый список карточек. Каждая карточка содержит иконку категории, название, описание, награды в опыте и золоте, статус и доступное действие.

Категории заданий помогают ребенку быстро понять тип поручения: учеба, дом, чтение, здоровье, спорт, гигиена, помощь семье или пользовательская категория. Награды отображаются с иконками опыта и золота. Статус может показывать, назначено ли задание, отправлено ли оно на проверку, одобрено, отклонено или уже получено.

Логика действий зависит от статуса. Назначенное или отклоненное задание можно отправить на проверку. Отправленное задание ожидает решения родителя. Одобренное задание позволяет забрать награду. Полученное задание становится завершенным. Такая логика помогает ребенку понимать, на каком этапе находится каждое поручение.

Визуальная стилизация доски имеет методическое значение. Если ребенок видит домашнюю обязанность как карточку приключения, задача воспринимается менее формально. Награда становится не абстрактным обещанием, а частью игровой экономики. Родитель при этом сохраняет контроль над подтверждением выполнения.

2.4 Синхронизация заданий между Unity и Web

Синхронизация заданий является центральным механизмом новой версии. Она связывает два пользовательских интерфейса и превращает проект в совместную систему. Родитель создает квест в веб-приложении,

после чего документ появляется в Firestore в подколлекции children/{childId}/quests. Unity-клиент при загрузке доски приключений получает этот список через FirebaseRestClient.

Когда ребенок отправляет задание на проверку, Unity выполняет PATCH-запрос к документу квеста. Статус меняется на submitted, добавляется комментарий ребенка и время отправки. После этого родитель может увидеть задание в разделе проверки веб-приложения. Если результат принят, родитель меняет статус на approved. Если требуется доработка, статус становится rejected, а комментарий родителя объясняет причину.

После одобрения ребенок может забрать награду. Unity-клиент вызывает claimQuestReward. В production-логике эта операция должна проходить через Cloud Functions, где проверяется статус квеста и начисляется награда. В MVP-режиме предусмотрен fallback: прямое обновление статуса и добавление награды в gameState. Так можно тестировать полный пользовательский путь еще до окончательного деплоя функций.

Синхронизация построена вокруг общей модели статусов. Это снижает риск расхождений между клиентами. Веб-приложение и Unity используют одинаковые смысловые состояния задания, пусть и представлены они разными языками программирования: TypeScript на стороне веба и C# на стороне Unity.

Для дальнейшего развития можно добавить realtime-обновления в Unity через периодическую загрузку или отдельный listener-слой. На текущем этапе достаточно ручной или экранной загрузки списка квестов, поскольку основной целью было подтвердить возможность обмена данными между приложениями.

2.5 Система наград и игровой прогресс

Игровой прогресс является мотивационной основой проекта. Ребенок получает опыт и золото как за игровые действия, так и за реальные задания. Это объединяет две активности в одну систему развития героя. Опыт влияет

на уровень, уровень открывает новые возможности, золото может использоваться для будущих покупок предметов, улучшений или косметических элементов.

В Unity-клиенте GameStateService отвечает за загрузку, сохранение и изменение игрового состояния. При получении награды опыт и золото добавляются к текущему состоянию. LevelService проверяет, достиг ли игрок нового уровня. После изменения сведения сохраняются в Firestore, если клиент авторизован, или остаются в локальном dev-состоянии при тестировании.

Боевая система также связана с прогрессом. BattleService создает боевую сессию, рассчитывает урон, критические попадания, комбо и награду за победу. Это сохраняет игровую самостоятельность проекта: ребенок может играть и получать награды не только за реальные задания, но и за прохождение игровых локаций. Однако реальные задания остаются более значимым источником прогресса, поскольку они связывают игру с поведением вне экрана.

Такой баланс важен. Если награды выдаются только за реальные задания, приложение может восприниматься как скрытый список обязанностей. Если награды выдаются только за игру, связь с воспитательной целью теряется. Комбинация игровых боев и реальных квестов делает систему более устойчивой: у ребенка есть развлечение, но самый ценный прогресс связан с полезными действиями.

В дальнейшем систему наград можно расширить предметами, коллекциями, косметическими элементами, улучшениями героя, ежедневными сериями и событиями. Текущая архитектура уже содержит InventoryService, EquipmentService и модели предметов, что создает основу для такого развития.

2.6 Импорт и организация графических ресурсов

Для повышения качества визуального оформления в проект была добавлена отдельная графическая база. Изображения были распределены по смысловым категориям: фоны, враги, эффекты, герой, иконки, предметы, карта, панели, портреты, квесты и городские элементы. Часть изображений была нарезана на отдельные спрайты, после чего на их основе были созданы префабы.

Организация ресурсов имеет практическое значение. Если графика лежит хаотично, дальнейшее развитие проекта быстро усложняется: разработчик тратит время на поиск нужного изображения, а экраны начинают использовать разные стилистические элементы. Разделение по папкам и единая ArtLibrary позволяют централизованно управлять ресурсами.

Враги получили привязку к спрайтам через конфигурационные объекты. Это означает, что визуальное представление врага можно менять без переписывания боевой логики. Аналогично фоны локаций выбираются на основании идентификатора подземелья или сцены. Такой подход соответствует data-driven-разработке: игровые сущности описываются данными, а код использует эти сведения для построения интерфейса.

Для UI были подготовлены панели, карточки, иконки и элементы доски. Они применяются на экранах входа, карты и города. Благодаря этому интерфейс перестал быть набором одноцветных прямоугольников и получил единую визуальную систему.

В дальнейшем графическую базу можно расширить анимациями, состояниями кнопок, эффектами получения награды, всплывающими подсказками и визуальными реакциями на действия ребенка. Но уже текущая переработка значительно повышает воспринимаемое качество приложения.

2.7 Адаптивность интерфейса

Проблема адаптивности стала заметной при проверке проекта на разных соотношениях сторон. Интерфейс, который приемлемо выглядел на

экране редактора или ПК, мог плохо отображаться на телефоне. Особенно страдали начальный экран и карта: фон мог растягиваться, элементы располагались неудобно, а часть пространства использовалась неэффективно.

Для решения была добавлена система SafeAreaRoot и SafeAreaFitter. Каждый основной экран создает корневой контейнер, который учитывает безопасную область экрана. Это важно для мобильных устройств с вырезами, скругленными углами и системными панелями. UI-элементы размещаются внутри этой области и не попадают под системные зоны.

Фоны теперь отображаются через SpriteBackground и AspectFillImage. Изображение не растягивается по ширине и высоте независимо, а масштабируется с сохранением пропорций. Лишняя часть может быть обрезана, но сцена не деформируется. Такой подход стандартен для полноэкранных игровых фонов и дает более профессиональный результат.

CanvasScaler был настроен на масштабирование относительно портретной базовой резолюции. Это помогает сохранять размер интерфейсных элементов на мобильных экранах. Для ландшафтного режима используется соответствующее соотношение matchWidthOrHeight, чтобы элементы не становились чрезмерно большими или маленькими.

Адаптивность особенно важна для детского приложения. Ребенок не должен бороться с мелкими кнопками, перекрытиями или нечитаемым текстом. Интерфейс должен быть понятным сразу, без необходимости объяснять, куда нажимать.

2.8 Исправление проблем интеграции и тестирования

В процессе интеграции были выявлены типичные проблемы, возникающие при соединении Unity, нового Input System и веб-инфраструктуры. Одна из ошибок была связана с использованием UnityEngine.Input при включенном Input System package. В проекте был заменен старый StandaloneInputModule на InputSystemUIInputModule, а для desktop-тестирования были скорректированы обращения к вводу. Это

позволило вводить логин и пароль на ПК и тестировать UI без мобильной сборки.

При работе с Next.js возникала ошибка Turbopack, связанная с записью endpoint и отсутствием найденного пакета Next.js в контексте сборщика. Для обхода в dev-скрипте был использован запуск через webpack. Это позволило продолжить тестирование родительского веб-приложения без блокировки разработки.

Кроме этого, были выявлены ошибки доступа Firestore: Missing or insufficient permissions. Их причина заключалась в несоответствии правил безопасности, ролей пользователей и операций, которые выполнялись клиентами. Firestore rules были доработаны так, чтобы родитель мог создавать и управлять заданиями своего ребенка, а детский Unity-клиент мог выполнять ограниченные операции над своими заданиями: отправлять на проверку и забирать одобренную награду.

Отдельная проблема была связана с тем, что сведения Unity-входа ребенка после создания не были явно видны родителю. В веб-приложении были добавлены поля и раздел, где родитель может увидеть email и тестовый пароль для входа ребенка в Unity. Это важно для MVP, потому что без видимого канала передачи учетных данных тестирование связки становится неудобным.

Проверка Unity-клиента выполнялась через сборку C#-проекта и запуск в редакторе. После внесения изменений проект собирался без ошибок. Это подтверждает корректность C#-кода, хотя полноценное пользовательское тестирование на мобильном устройстве должно быть отдельным этапом.

2.9 Изменение качества продукта по сравнению с первой версией

Главное отличие новой версии заключается в переходе от локального игрового прототипа к связанной системе. В первой версии проект демонстрировал игровую идею. В новой версии он демонстрирует

продуктовую логику: есть ребенок, родитель, общие сведения, статусы, награды и интерфейсы для двух ролей.

Функционально проект стал богаче. Появилась родительская веб-панель, настройка семьи, Unity-логин ребенка, создание заданий, просмотр квестов, проверка отправленных заданий, Firestore rules и Cloud Functions-слой для будущей production-логики. Unity-клиент научился работать с облачной моделью, а не только с локальными демонстрационными данными.

Визуально проект стал более цельным. Стартовый экран получил образ входа в приключение. Карта перестала быть набором кнопок и стала пространством с объектными локациями. Доска заданий получила стилизацию под гильдейские поручения. Враги, предметы, фоны и панели были организованы как полноценная графическая база.

Технически проект стал более расширяемым. Модели данных разделены по сущностям, сервисы отвечают за отдельные области, а UI использует общие вспомогательные методы. Firestore-модель позволяет добавлять новые поля и подколлекции, а Unity-структура позволяет добавлять новые сцены, локации и типы наград.

С точки зрения пользовательской ценности проект приблизился к реальному сценарию. Родитель может управлять заданиями, ребенок видит их в игре, выполнение проходит через проверку, а награда влияет на прогресс. Это и является ключевым улучшением по сравнению с первой версией.

2.10 Проверяемые пользовательские сценарии

Для оценки работоспособности улучшенной версии были выделены основные пользовательские сценарии. Первый сценарий связан с родителем: вход в веб-приложение, создание семьи, добавление ребенка, создание Unity-входа и просмотр данных для передачи ребенку. Второй сценарий связан с созданием задания: родитель выбирает ребенка, заполняет название, описание, категорию и награды, после чего задание появляется в Firestore.

Третий сценарий относится к ребенку: вход в Unity-приложение с использованием учетных данных, загрузка профиля, переход на карту, открытие города и просмотр доски приключений. На доске должны отображаться задания, созданные родителем. Ребенок должен понимать статус каждого задания и видеть доступное действие.

Четвертый сценарий проверяет отправку задания на проверку. Ребенок выбирает назначенное задание, отправляет его, после чего статус меняется на submitted. В веб-приложении родитель должен увидеть такое задание в разделе проверки.

Пятый сценарий проверяет одобрение и получение награды. Родитель одобряет задание, Unity отображает его как готовое к получению награды, ребенок нажимает соответствующую кнопку, после чего опыт и золото добавляются к игровому состоянию.

2.11 Техническая проверка

Техническая проверка включала сборку Unity C#-части, проверку структуры файлов, проверку корректности импортированных ресурсов и анализ основных ошибок интеграции. После внесения изменений проект собирался без ошибок компиляции. Это важно, поскольку переработка затронула несколько экранов, общие UI-утилиты и интеграционные сервисы.

Отдельно проверялась совместимость с Input System. Ошибка обращения к UnityEngine.Input при активном новом Input System была

устранена. Пользовательский ввод в поля логина и пароля стал возможен при тестировании на ПК.

В веб-приложении была проверена возможность локального запуска через Next.js с webpack-режимом вместо Turbopack. Это позволило обойти ошибку dev-сборщика и продолжить тестирование страниц.

Firestore rules были приведены к модели ролей. Ошибки прав доступа при создании или изменении квестов были связаны с тем, что база должна явно понимать, кто выполняет операцию: родитель или ребенок. После уточнения правил стали возможны операции, соответствующие роли пользователя.

2.12 Ограничения текущей версии

Несмотря на значительное развитие проекта, текущая версия остается MVP и имеет ограничения. Во-первых, часть операций в веб-приложении и Unity временно может выполняться прямыми обновлениями Firestore. Для production-версии следует перенести одобрение, отклонение и выдачу награды в Cloud Functions.

Во-вторых, Unity-клиент пока не использует полноценную систему realtime-обновлений. Задания загружаются при открытии соответствующего экрана или через явный запрос. В будущем можно добавить автоматическое обновление списка заданий при изменениях в Firestore.

В-третьих, визуальная система была существенно улучшена, но требует дальнейшей художественной полировки: анимации, плавные переходы, эффекты наград, состояния кнопок, звуковые реакции и микровзаимодействия.

В-четвертых, система аналитики пока находится в заготовочном состоянии. Для родителя полезно показывать не только количество заданий, но и динамику выполнения, категории, регулярность, время активности и общий прогресс ребенка.

В-пятых, требуется полноценное мобильное тестирование на устройствах с разными разрешениями. Адаптивные механизмы уже добавлены, но качество интерфейса окончательно оценивается только на реальных экранах.

2.13 Итоговые результаты

В результате работы исходный прототип был преобразован в более зрелую систему. Был создан и подключен родительский веб-интерфейс, реализована общая модель данных, добавлена авторизация ребенка, настроена синхронизация заданий, доработаны правила доступа, улучшена визуальная часть Unity-клиента и добавлена адаптивность.

Проект получил понятное разделение ролей. Родитель действует через веб-панель, ребенок – через игру. Это соответствует реальному поведению пользователей и позволяет каждому интерфейсу быть оптимизированным под свою задачу.

Игровая часть стала более выразительной. Карта теперь выглядит как игровая карта, а не как меню. Задания представлены как карточки приключений. Фоны и панели формируют единый стиль. Это повышает эмоциональную привлекательность приложения.

Системная часть стала более практичной. Firestore хранит общие сведения, Authentication разделяет пользователей, а Cloud Functions задают направление для безопасной серверной логики. Unity-клиент и веб-приложение используют совместимую модель сущностей.

В итоге новая версия проекта демонстрирует не только игровую идею, но и работающий подход к созданию семейного геймифицированного продукта.

3 Расширение системы: геолокационные квесты, аналитика и экспертные рекомендации

3.1 Обоснование направлений дальнейшей модернизации

Текущая версия проекта уже решает основную задачу модернизации исходного Unity-прототипа: игровой клиент ребенка связан с родительским веб-приложением, задания создаются во внешнем интерфейсе, сохраняются в облачной базе данных и могут отображаться в игровом приложении. Однако разработанная система остается основой для дальнейшего развития. На следующем этапе важно не только увеличивать количество игровых локаций и предметов, но и усиливать именно прикладную ценность проекта как инструмента родительского взаимодействия.

Главным направлением развития является переход от простой выдачи заданий к более интеллектуальной модели сопровождения ребенка. Родитель должен не только вручную создавать поручения, но и получать подсказки о том, какие задания ребенок выполняет охотнее, какие категории вызывают затруднения, как меняется регулярность выполнения и какие типы наград лучше поддерживают интерес. Для этого требуется аналитический модуль и система рекомендаций, работающая на основе истории выполненных квестов.

Вторым важным направлением является реализация геолокационных заданий. В текущей версии задание подтверждается в основном через статус и комментарий ребенка. В перспективе можно добавить квесты, связанные с посещением реальных мест: школы, спортивной секции, библиотеки, кружка, парка или иной точки, выбранной родителем. Такой подход расширяет идею связи реальных действий с игровым прогрессом: ребенок получает награду не только за домашние дела, но и за регулярное посещение полезных мест.

Третьим направлением является усиление доверенной серверной логики. Чем больше игровых наград, геолокационных условий и аналитических выводов появляется в системе, тем меньше критических операций должно выполняться непосредственно на клиенте. Поэтому одобрение заданий, начисление наград, учет посещений и построение рекомендаций целесообразно выполнять через Cloud Functions или отдельный backend-слой. Это снижает риск некорректного изменения данных и позволяет централизованно контролировать бизнес-правила.

3.2 Расширение модели реального квеста

Для поддержки будущих функций необходимо расширить модель реального задания. В первой версии квест может содержать название, описание, категорию, статус, награду в опыте и золоте, а также комментарии ребенка и родителя. Для аналитики, рекомендаций и геолокации этого недостаточно. Нужно хранить не только итоговый статус, но и контекст выполнения: сложность, повторяемость, ожидаемую продолжительность, дедлайн, количество напоминаний, тип подтверждения, временные метки и связанные метрики поведения.

Расширенная модель не должна ломать существующий жизненный цикл. Базовые статусы assigned, submitted, approved, rejected, claimed и cancelled сохраняются, но дополняются новыми полями. Так можно постепенно добавлять функциональность без полной переработки Unity-клиента и веб-приложения. Состав полей расширенной модели задания представлен в таблице 3.1.

Таблица 3.1 – Поля расширенной модели задания

Группа полей	**Примеры полей**	**Назначение**
Основные сведения	title, description, category, difficulty, recurrenceType	Описывают Содержание задания, его категорию, сложность и повторяемость.
Награды	rewardXp, rewardGold, rewardItems, bonusMultiplier	Позволяют настраивать мотивационное подкрепление за

Группа полей	Примеры полей	Назначение
		выполнение.
Сроки	createdAt, dueAt, submittedAt, approvedAt, claimedAt	Используются для аналитики скорости выполнения и контроля дедлайнов.
Подтверждение	verificationType, proofRequired, proofImageUrl, childComment	Определяют способ проверки: комментарий, фото, ручное подтверждение, геолокация.
Геолокация	locationQuest, targetPlaceId, latitude, longitude, radiusMeters, requiredVisits	Хранят параметры задания, связанного с посещением точки.
Аналитика	viewCount, reminderCount, rejectionCount, completionDurationHours	Формируют признаки для рекомендаций и отчетов родителя.
Рекомендации	recommendationScore, recommendationReason, modelVersion	Позволяют объяснить, почему система предлагает или не предлагает квест.

Такое расширение модели особенно важно для дальнейшего машинного анализа. Простого факта выполнения недостаточно, поскольку он не показывает, было ли задание удобным, интересным или слишком сложным. Например, два задания могут быть одобрены, но одно выполнено ребенком в тот же день без напоминаний, а второе – через несколько дней после повторного отклонения. Для родителя эти случаи имеют разный смысл, и система должна различать их при формировании рекомендаций.

3.3 Общий жизненный цикл расширенного задания

Расширенный жизненный цикл задания должен учитывать не только назначение и подтверждение, но и сбор аналитических событий. При создании квеста родитель указывает не только текст и награду, но и тип задания: обычное, повторяемое, геолокационное, учебное, бытовое, спортивное или комбинированное. После публикации квест появляется у ребенка в Unity-клиенте и становится частью игрового интерфейса.

На стороне ребенка фиксируются события просмотра, принятия, отправки на проверку, получения отказа или награды. Эти события не должны превращаться в избыточное слежение, но должны давать достаточно данных для понимания динамики. Например, полезно знать, что ребенок регулярно открывает квесты категории «чтение», быстро отправляет их на проверку и почти не получает отказов. Такая категория может быть рекомендована родителю как подходящая для дальнейшего развития привычки.

Для геолокационного задания жизненный цикл дополняется проверкой посещения. Ребенок может нажать кнопку «Отметиться» в нужной точке или приложение может получить событие входа в геозону. После этого создается запись check-in. Когда количество подтвержденных посещений достигает требуемого значения, задание автоматически переводится в submitted или approved в зависимости от выбранной родителем модели проверки.

При этом окончательное начисление награды должно оставаться серверной операцией. Клиент может инициировать действие, но Cloud Function проверяет статус, принадлежность задания ребенку, отсутствие повторного начисления и только после этого меняет gameState и rewardLedger. Firebase Callable Functions поддерживают вызов функций из клиентских приложений, включая Unity, что позволяет использовать их для доверенных операций [12].

3.4 Связь планируемых функций с текущей архитектурой

Планируемые функции не требуют замены существующего стека. Unity остается игровым клиентом ребенка, Next.js – веб-интерфейсом родителя, Firestore – основным хранилищем, а Cloud Functions – слоем серверной логики. Firestore хорошо подходит для такой модели, поскольку хранит сведения в документах, коллекциях и подколлекциях, что позволяет естественно выделять у ребенка подколлекции quests, checkIns, analyticsDaily и recommendations [10].

На уровне Unity потребуется добавить LocationQuestService, AnalyticsEventService и RecommendationPreviewService. Первый будет отвечать за геолокационные проверки, второй – за отправку событий активности, третий – за отображение рекомендованных или приоритетных заданий на доске приключений. На стороне веб-приложения потребуется добавить экран настройки геолокационного задания, аналитическую вкладку и блок рекомендаций при создании нового квеста.

На уровне Cloud Functions появятся функции registerQuestEvent, validateLocationCheckIn, aggregateDailyAnalytics, generateQuestRecommendations и claimQuestReward. В MVP-версии некоторые из этих операций могут выполняться проще, однако итоговая архитектура должна предусматривать перенос критической логики на серверную сторону.

3.5 Назначение и пользовательский сценарий геолокационного задания

Геолокационный квест представляет собой задание, выполнение которого связано с посещением определенного реального места. В отличие от обычного задания, где ребенок сам сообщает о выполнении, геолокационный квест использует координаты устройства или событие входа в геозону как дополнительное подтверждение. Такой механизм может применяться для задач вроде «посетить школу 5 раз за неделю», «прийти на тренировку», «посетить библиотеку», «погулять в парке» или «дойти до кружка».

Родительский сценарий начинается с создания задания в веб-панели. Родитель выбирает тип «геолокационный квест», указывает название места, координаты, радиус допустимой зоны, количество посещений, период действия, расписание и награду. Для упрощения интерфейса координаты могут задаваться через карту, поиск места или ручной ввод. После сохранения квест появляется в Unity-клиенте ребенка.

Детский сценарий должен быть максимально прозрачным. Ребенок видит, что квест связан с посещением места, видит прогресс посещений и понимает, когда награда станет доступной. Если используется активная проверка, ребенок нажимает кнопку «Отметиться», а приложение проверяет, находится ли он внутри разрешенного радиуса. Если используется автоматический режим, приложение получает событие входа в геозону и обновляет прогресс без постоянного ручного действия.

Для детского приложения предпочтительно начинать с активной проверки посещения. Она проще технически, меньше расходует батарею и лучше соответствует принципу минимизации данных: приложение не хранит маршрут ребенка, а фиксирует только факт посещения точки при конкретного задания. Автоматический режим можно рассматривать как

следующий этап, когда будет готова политика приватности, понятный интерфейс согласия и серверная защита от повторных срабатываний.

3.6 Выбор технического подхода и SDK

В Unity базовый доступ к координатам устройства может быть реализован через LocationService. В документации Unity указывается, что после вызова Start у сервиса можно получать последние сведения местоположения через Input.location.lastData, но сервис начинает выдавать координаты не мгновенно, поэтому необходимо проверять его статус [17]. Такой вариант подходит для активной проверки посещения, когда ребенок сам нажимает кнопку отметки.

Для Android-приложения также необходимо учитывать разрешения на доступ к местоположению. Документация Android разделяет приблизительное и точное местоположение: ACCESS_COARSE_LOCATION дает приблизительную оценку, а ACCESS_FINE_LOCATION позволяет получить более точные координаты [19]. Для квестов вроде «посетить школу» чаще всего требуется точность выше приблизительной, но в интерфейсе нужно объяснять родителю и ребенку, зачем требуется такое разрешение.

Если требуется автоматическое отслеживание входа в зону, более подходящим вариантом является Android Geofencing API. Официальная документация Android описывает создание и мониторинг геозон, а также обработку переходов через BroadcastReceiver [18]. Для Unity такой функционал удобнее реализовывать через небольшой нативный Android-плагин на Kotlin или Java, который будет работать с GeofencingClient и передавать результат в Unity через AndroidJavaObject или UnitySendMessage.

В итоге целесообразно предусмотреть два режима работы. Первый режим – активный check-in на Unity LocationService. Он используется в MVP и работает только при открытом приложении. Второй режим – автоматический geofence на Android через Google Play services. Он

используется как расширение, когда требуется фиксировать посещение без ручного нажатия. Сравнение способов геолокационной проверки представлено в таблице 3.2.

Таблица 3.2 – Сравнение способов геолокационной проверки

Подход	**Преимущества**	**Ограничения**	**Рекомендуемое применение**
Unity LocationService	Простая интеграция в Unity; достаточно для ручной отметки; легче тестировать.	Работает в основном при активном приложении; требует ожидания статуса сервиса; не является полноценным background-geofence.	MVP геолокационных квестов: кнопка «Отметиться».
Android Geofencing API	Позволяет отслеживать вход/выход/пребывание в геозоне; лучше подходит для автоматического сценария.	Требует нативного Android-плагина, разрешений и аккуратной работы с батареей и приватностью.	Production-этап: автоматическое подтверждение посещений.
Ручное подтверждение родителем	Не требует GPS; хорошо для чувствительных ситуаций.	Меньше автоматизации; зависит от проверки родителя.	Резервный вариант, если геолокация отключена.
QR-код/NFC	Подтверждает присутствие в конкретном месте без постоянного GPS.	Нужна физическая метка или код в месте посещения.	Альтернатива для школ, кружков, домашних зон.

3.7 Реализация активной проверки посещения в Unity

Активная проверка посещения является наиболее подходящей для первого этапа. В интерфейсе квеста появляется кнопка «Отметиться». После нажатия Unity-клиент запрашивает разрешение на использование местоположения, запускает LocationService, ожидает перехода сервиса в состояние Running, получает координаты и сравнивает их с координатами целевой точки.

Сравнение выполняется через расчет расстояния между двумя географическими координатами. Для этого можно использовать формулу гаверсинусов или встроенную утилиту, если она будет вынесена в LocationMathService. Если расстояние меньше или равно radiusMeters, создается событие посещения. Если расстояние превышает радиус, ребенок получает понятное сообщение: «Вы пока не в нужной зоне».

Упрощенный алгоритм работы активной проверки может быть описан следующим образом:

1) пользователь открывает карточку геолокационного квеста;

2) Unity проверяет, есть ли разрешение на использование местоположения;

3) если разрешения нет, отображается системный запрос;

4) запускается LocationService с заданной точностью;

5) клиент ожидает получения координат с ограничением по времени;

6) вычисляется расстояние до целевой точки;

7) при успешной проверке вызывается Cloud Function validateLocationCheckIn;

8) сервер проверяет отсутствие повторной отметки за тот же период и увеличивает счетчик посещений.

Даже если расстояние вычисляется на клиенте, окончательное подтверждение желательно выполнять на сервере. Клиент передает координаты, точность, время и идентификатор задания, а сервер сверяет эти сведения с параметрами квеста. Это не устраняет все возможные попытки подмены геолокации, но снижает риск случайных ошибок и двойного учета посещений.

3.8 Реализация автоматического геозонирования на Android

Автоматическое геозонирование является более сложным этапом. Для него потребуется нативный Android-плагин, так как Unity LocationService сам по себе предназначен для получения текущего местоположения, а не для

регистрации системных геозон. Плагин должен принимать список активных геолокационных квестов, создавать для них geofence-объекты и регистрировать обработчик событий.

На Android геозона задается координатами центра, радиусом, временем жизни и типами переходов. В документации Google Play services Geofence описывается как географическая область, за пересечением границы которой может наблюдать геолокационный сервис [18]. Для проекта будут полезны события ENTER и DWELL. ENTER показывает вход в зону, а DWELL может использоваться для подтверждения, что ребенок пробыл в месте некоторое время, а не просто проехал мимо.

Компоненты автоматического режима:

1) UnityLocationBridge – C#-класс для связи Unity и Android;

2) AndroidGeofencePlugin – Kotlin/Java-класс, создающий geofence;

3) GeofenceBroadcastReceiver – получает события от системы;

4) LocationQuestService – отправляет событие в Firebase;

5) validateLocationCheckIn – серверная функция, проверяющая событие и обновляющая прогресс.

Автоматический режим следует включать только для квестов, где он действительно нужен. Например, для регулярного посещения школы или секции он может быть удобен, но для домашних поручений и чтения не нужен. Это важно для экономии заряда и соблюдения принципа минимизации данных.

3.9 Модель данных Firestore для геолокационных квестов

Для хранения геолокационных квестов можно использовать существующую подколлекцию children/{childId}/quests/{questId}, добавив в документ блок locationRule. Такой подход удобен, потому что геолокационное задание остается обычным квестом с теми же статусами и наградами. Дополнительная информация хранится только у тех заданий, где

verificationType равен location. Состав полей геолокационного задания представлен в таблице 3.3.

Таблица 3.3 – Поля геолокационного задания

Поле	**Тип**	**Описание**
verificationType	string	Тип подтверждения: manual, photo, location, hybrid.
locationRule.placeName	string	Название места, понятное родителю и ребенку.
locationRule.latitude	number	Широта целевой точки.
locationRule.longitude	number	Долгота целевой точки.
locationRule.radiusMeters	number	Допустимый радиус посещения.
locationRule.requiredVisits	number	Сколько посещений нужно для выполнения задания.
locationRule.minDwellMinutes	number	Минимальное время пребывания в зоне, если используется автоматический режим.
locationRule.schedule	object	Ограничения по дням недели и времени.
progress.visitsCount	number	Количество подтвержденных посещений.
progress.lastCheckInAt	timestamp	Последнее успешное посещение.

Отдельную подколлекцию целесообразно использовать для событий посещения: children/{childId}/quests/{questId}/checkIns/{checkInId}. В ней хранятся дата, способ подтверждения, точность, округленные координаты, статус проверки и ссылка на серверную функцию, которая обработала событие. Точные координаты лучше не хранить без необходимости. Для аналитики обычно достаточно факта посещения, времени и результата проверки.

Пример структуры check-in события:

checkInId: автоматический идентификатор

questId: идентификатор задания

childId: идентификатор ребенка

createdAt: время события

source: active_button или android_geofence

accuracyMeters: точность координат

distanceToTargetMeters: расстояние до целевой точки

validated: true или false

validationReason: inside_radius, outside_radius, duplicate_day, outside_schedule

3.10 Безопасность, приватность и ограничения геолокационного модуля

Геолокация является чувствительной частью проекта, особенно потому что приложение ориентировано на ребенка. Поэтому модуль должен проектироваться не как система постоянного контроля, а как механизм подтверждения конкретного квеста. В интерфейсе должно быть явно указано, какое место связано с заданием, сколько посещений требуется и когда приложение проверяет координаты.

Основные принципы безопасности:

1) геолокационные квесты выключены по умолчанию;

2) родитель явно создает конкретную точку и цель посещения;

3) ребенок видит, что квест использует геолокацию;

4) система не хранит маршрут и историю перемещений;

5) сохраняется только факт посещения при задания;

6) доступ к этим ограничивается Firestore rules и серверными проверками;

7) повторные отметки за один период не должны начислять награду повторно.

Для MVP наиболее безопасным вариантом является активная проверка по кнопке. В этом случае координаты запрашиваются только в момент выполнения действия ребенком. Автоматическое геозонирование следует

реализовывать как расширение и сопровождать отдельными настройками согласия и уведомлениями.

Техническим ограничением геолокации является погрешность. Внутри зданий координаты могут быть неточными, а в плотной городской застройке возможны отклонения. Поэтому радиус геозоны не должен быть слишком маленьким. Для школы, парка или секции можно использовать радиус, который компенсирует погрешность, но не делает зону слишком широкой. В интерфейсе родителя нужно добавить подсказку о выборе радиуса.

3.11 Проверка работоспособности геолокационных сценариев

Тестирование геолокационных квестов должно включать несколько уровней. На первом уровне проверяется логика без реального GPS: разработчик задает фиктивные координаты в dev-режиме Unity и убеждается, что расстояние считается правильно. На втором уровне используется Android-устройство или эмулятор с mock location. На третьем уровне проверяется реальный сценарий посещения точки. Основные сценарии проверки геолокационных квестов представлены в таблице 3.4.

Таблица 3.4 – Сценарии проверки геолокационных квестов

Сценарий проверки	**Ожидаемый результат**
Разрешение на геолокацию не выдано	Unity показывает понятное сообщение и предлагает разрешить доступ.
Сервис геолокации выключен	Приложение предлагает включить геолокацию в настройках устройства.
Ребенок находится внутри радиуса	Создается check-in, счетчик посещений увеличивается.
Ребенок находится вне радиуса	Появляется сообщение о невозможности отметиться.
Повторная отметка в тот же день	Сервер не увеличивает счетчик повторно.
Отметка вне разрешенного расписания	Событие фиксируется как отклоненное.
Неустойчивая точность GPS	Приложение просит повторить проверку или расширить радиус.

3.12 Реализованный аналитический модуль и задачи родителя

В обновленной версии проекта реализован аналитический модуль родительской веб-панели. Он рассматривает задания не только как отдельные записи Firestore, но и как источник данных о динамике выполнения: какие категории ребенок чаще завершает, сколько времени занимает выполнение, как часто требуется повторная проверка и насколько регулярно забирается награда.

Задача аналитики состоит не в скрытом наблюдении за ребенком, а в поддержке решения родителя. Интерфейс показывает агрегированные показатели и помогает выбирать посильные задания, корректировать сложность и избегать чрезмерной нагрузки.

В веб-приложении доступны сводные показатели по дням и категориям, а также рекомендации для формы создания квеста. Такой подход превращает историю выполнения в практический инструмент настройки мотивации.

3.13 Событийная модель и собираемые показатели

Для аналитики реализована событийная модель. Значимые переходы жизненного цикла фиксируются отдельными документами questEvents: создание и просмотр задания, отправка на проверку, одобрение, отклонение, отмена и получение награды.

События создаются в Unity-клиенте и родительском веб-приложении в тех же точках, где изменяется состояние квеста. Благодаря этому аналитический след связан с реальным пользовательским действием, а не восстанавливается только по итоговому статусу.

Для каждого события сохраняются идентификаторы семьи, ребенка и квеста, роль инициатора, клиент-источник, категория, сложность, награда, локальная дата и время, а при наличии – длительность выполнения и переход статуса. Состав основных событий представлен в таблице 3.5.

Таблица 3.5 – События жизненного цикла квеста

Тип события	**Источник**	**Что фиксируется**	**Назначение для анализа**
quest_created	Web	Родитель создал задание	Оценка количества назначенных квестов и нагрузки
quest_viewed	Unity	Ребенок открыл карточку задания	Понимание интереса и реакции на задание
quest_submitted	Unity	Ребенок отправил выполнение на проверку	Расчет времени выполнения и активности
quest_approved	Web	Родитель одобрил результат	Оценка качества выполнения
quest_rejected	Web	Родитель отклонил результат	Выявление сложных или плохо сформулированных заданий
quest_claimed	Unity	Ребенок забрал награду	Фиксация завершенного мотивационного цикла
reminder_sent	Web/System	Ребенку отправлено напоминание	Оценка необходимости внешней поддержки
location_check_in_success	Unity/System	Подтверждено посещение точки	Анализ геолокационных квестов

Каждое событие содержит familyId, childId, questId, тип события, роль автора действия, источник события, категорию квеста, сложность, награду в опыте и золоте, предыдущий и новый статус, дату и время события. Для геолокационных заданий дополнительно может храниться расстояние до целевой точки и результат проверки посещения.

На основе событий рассчитываются показатели: доля завершенных заданий, доля одобрений, частота отклонений, среднее время от назначения до отправки, среднее время от одобрения до получения награды, количество напоминаний, соотношение просмотренных и отправленных заданий, успешность по категориям и уровень нагрузки в конкретный день.

3.14 Агрегация данных и ежедневные витрины статистики

На основе questEvents сервис analyticsService формирует документы analyticsDaily и analyticsByCategory. Первая коллекция содержит показатели по датам, вторая – накопленную статистику по категориям заданий.

В текущей конфигурации бесплатного тарифа Firebase пересчет выполняется в веб-клиенте родителя и разрешается правилами Firestore только для ребенка из той же семьи. Исходный код Cloud Functions подготовлен для переноса агрегации в доверенную серверную среду после развертывания проекта на тарифе, поддерживающем функции.

Витрины не заменяют исходные события: они содержат подготовленные показатели для быстрого отображения интерфейса. Основные показатели приведены в таблице 3.6.

Таблица 3.6 – Основные аналитические показатели

Показатель	**Источник расчета**	**Как используется**
completionRate	claimed / assigned	Показывает, какая доля заданий доводится до награды
approvalRate	approved / submitted	Оценивает качество отправленных результатов
rejectionRate	rejected / submitted	Выявляет задания, которые часто требуют доработки
avgSubmitHours	assignedAt и submittedAt	Показывает скорость выполнения
viewToSubmitRate	submitted / viewed	Оценивает переход от интереса к действию
reminderNeedRate	reminder_sent / assigned	Показывает зависимость от напоминаний
activeQuestLoad	активные задания за день	Позволяет не перегружать ребенка

Обобщенная схема аналитико-рекомендательного контура представлена на рисунке 3.1.



Рисунок 3.1 – Схема аналитико-рекомендательного контура

3.15 Оценка интереса ребенка к различным типам квестов

Интерес ребенка нельзя надежно определить только по одному признаку. Если ребенок быстро выполнил задание, это может означать высокую заинтересованность, но может быть связано и с простотой поручения. Если он открыл карточку, но не отправил результат, это может означать недостаточную мотивацию, слишком сложную формулировку или неподходящее время. Поэтому оценка интереса строится как совокупность нескольких признаков.

Для каждой категории заданий можно рассчитать относительный интерес. В него входят частота просмотра карточек, доля отправленных заданий после просмотра, доля одобренных результатов, скорость выполнения, количество повторных отказов, число напоминаний и динамика по неделям. Если категория стабильно выполняется быстро, редко отклоняется и не требует частых напоминаний, она считается подходящей для ребенка.

Отдельно учитывается эффект усталости. Даже полезная категория может начать выполняться хуже, если повторяется слишком часто.

Например, если задания по уборке назначаются почти каждый день, а completionRate падает, система должна не просто рекомендовать эту категорию как важную, а предупредить родителя о возможной перегрузке. Это делает аналитику более практичной: она показывает не только успехи, но и риски.

3.16 Представление аналитики в родительском веб-интерфейсе

В родительском веб-интерфейсе аналитика должна быть представлена в сжатом и понятном виде. Родитель не должен анализировать технические события вручную. Ему нужны готовые выводы: какие категории идут хорошо, какие требуют внимания, какие задания лучше назначать в ближайшее время и где стоит изменить сложность или награду.

Интерфейс аналитики целесообразно разделить на три уровня. Первый уровень – краткие карточки: всего назначено, выполнено, одобрено, отклонено, среднее время выполнения и текущая серия активности. Второй уровень – таблица по категориям, где видны completionRate, rejectionRate, avgSubmitHours и reminderNeedRate. Третий уровень – блок рекомендаций, сформированный экспертной системой.

Такой подход сохраняет простоту интерфейса, но делает систему значительно более полезной. Родитель видит не просто журнал действий, а получает инструмент анализа поведения и мотивации ребенка. На этом основании можно переходить к рекомендательному модулю.

3.17 Постановка задачи рекомендации

Рекомендательный модуль реализован как средство поддержки родителя при создании нового квеста. Он не заменяет решение взрослого, а оценивает выбранную категорию и предлагает сложность и размер награды с учетом накопленной статистики.

Практический результат представлен оценкой от 0 до 100, меткой «рекомендуется», «нейтрально» или «не рекомендуется», списком причин,

рисков и предлагаемых действий. Родитель может применить совет или сохранить собственные параметры задания.

Модуль использует сведения из questEvents, analyticsDaily и analyticsByCategory. При отсутствии истории система выдает нейтральную рекомендацию и стандартные параметры, поэтому форма остается работоспособной и для нового ребенка.

3.18 Сведения и признаки, используемые рекомендательной системой

Входными данными рекомендательной системы являются не только параметры нового задания, но и история выполнения похожих квестов. Для каждого ребенка система учитывает категорию задания, сложность, размер награды, тип подтверждения, срок выполнения, повторяемость, количество активных заданий в день, а также прошлые реакции ребенка на задания той же категории.

Ключевые признаки можно разделить на несколько групп. Первая группа описывает само задание: category, difficulty, rewardXp, rewardGold, verificationType, recurrenceType и dueAt. Вторая группа описывает поведение ребенка: viewCount, submittedCount, claimedCount, rejectionCount, reminderCount. Третья группа содержит временные признаки: avgSubmitHours, avgClaimHours, dayOfWeek, localHour и preferredTimeWindow. Четвертая группа отражает нагрузку: activeQuestLoad и sameCategoryAssignedLast7Days.

Наличие большого числа признаков делает систему более серьезной, потому что рекомендация строится не на одном факте выполнения, а на совокупности данных. Например, задание может иметь высокий completionRate, но при этом требовать слишком много напоминаний. В такой ситуации система не должна автоматически считать его удачным. Она может предложить родителю изменить формулировку, уменьшить объем или назначить задание в другое время.

Таблица 3.7 – Признаки рекомендательной системы

Группа признаков	**Примеры полей**	**Роль в рекомендации**
Параметры задания	category, difficulty, rewardXp, rewardGold	Определяют тип и мотивационную стоимость квеста
История выполнения	completionRate, approvalRate, rejectionRate	Показывают успешность похожих заданий
Поведенческая реакция	viewToSubmitRate, reminderNeedRate	Показывают интерес и необходимость внешней поддержки
Время	avgSubmitHours, localHour, dayOfWeek	Помогают учитывать удобное время выполнения
Нагрузка	activeQuestLoad, sameCategoryAssignedLast7Days	Защищают от перегрузки и однообразия
Геолокация	locationQuest, checkInResult, requiredVisits	Используются для заданий, связанных с посещением мест

3.19 Создание логической экспертной системы

На раннем этапе проекта наиболее подходящим вариантом является логическая экспертная система. Ее преимущество состоит в том, что она может работать даже при небольшом объеме накопленных данных. Для дерева решений или другой обучаемой модели нужна история большого числа выполненных заданий, а экспертная система может использовать заранее заданные правила и постепенно уточняться по мере развития продукта.

Экспертная система состоит из трех основных частей: базы знаний, набора входных признаков и механизма вывода. База знаний содержит правила вида "если условие выполняется, изменить оценку рекомендации и добавить объяснение". Входные признаки рассчитываются из аналитики ребенка. Механизм вывода последовательно применяет правила, суммирует их влияние и формирует итоговую оценку recommendationScore.

Базовая оценка рекомендации принимается равной 50 баллам. Каждое правило может повысить или снизить эту оценку. После применения всех правил значение ограничивается диапазоном от 0 до 100. Если результат выше 70, задание считается рекомендованным. Если оценка находится в

диапазоне от 40 до 69, задание считается нейтральным. Если оценка ниже 40, система предупреждает родителя, что задание лучше изменить или отложить.

Таблица 3.8 – Примеры правил экспертной системы

Правило	**Условие**	**Влияние**	**Объяснение для родителя**
Успешная категория	completionRate >= 0.75 и rejectionRate <= 0.15	+25	Похожие задания ребенок обычно доводит до награды
Быстрое выполнение	avgSubmitHours <= 24	+15	Задания этой категории выполняются быстро
Частые отклонения	rejectionRate >= 0.35	-30	Результаты часто требуют доработки
Низкий переход к отправке	viewedTotal >= 3 и viewToSubmitRate < 0.4	-20	Ребенок открывает задания, но редко отправляет их
Перегрузка дня	activeQuestLoad >= 5	-25	На день уже назначено много активных квестов
Усталость от категории	sameCategoryAssignedLast7Days >= 5 и completionRate < 0.6	-20	Однотипные задания начали выполняться хуже
Мало данных	assignedTotal < 3	0	Категорию стоит проверить через простое пробное задание
Эффективная награда	completionRate >= 0.7 и средняя награда умеренная	+10	Задания выполняются без завышения награды

Результат работы экспертной системы включает не только числовой score, но и список reasons, risks и suggestedActions. Например, если система рекомендует задание по чтению, она может объяснить: "похожие задания выполнялись быстрее одного дня", "отклонений почти не было", "категория показывает стабильную серию выполнения". Если система не рекомендует задание, она может указать: "в этой категории высокий процент отказов", "ребенок часто открывает карточку, но не отправляет результат", "сегодня уже назначено много активных заданий".

Такой формат делает экспертную систему объяснимой. Родитель видит не скрытый вывод программы, а понятную цепочку причин. Это особенно важно, поскольку система работает с поведением ребенка и не должна принимать решения непрозрачно.

Структура экспертной системы рекомендаций, связывающая признаки, базу правил и механизм вывода, показана на рисунке 3.2.

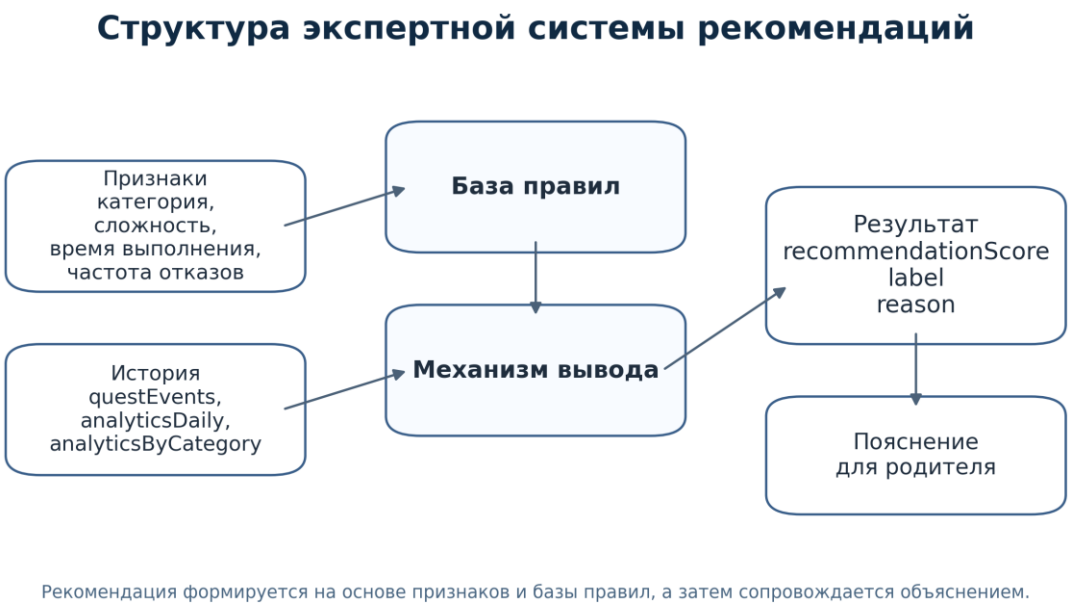


Рисунок 3.2 – Структура экспертной системы рекомендаций

3.20 Механизм работы экспертной системы в приложении

Экспертная система запускается при выборе категории в форме создания квеста. Сервис загружает агрегированную статистику ребенка и формирует набор признаков RecommendationFeatures.

Механизм вывода последовательно применяет правила базы знаний. Каждое сработавшее правило изменяет оценку и добавляет объяснение, риск либо практический совет. Итоговый QuestRecommendation содержит рекомендуемую сложность, награду, числовую оценку и текстовые основания.

Веб-интерфейс показывает результат рядом с формой создания задания. Дополнительно реализована коллекция готовых шаблонов, а

стандартная награда автоматически зависит от сложности: ручное изменение опыта и золота вынесено в расширенные настройки.

Unity-клиент получает признак recommended и recommendationScore вместе с квестом и может визуально выделить рекомендованное задание, не перегружая ребенка аналитическими подробностями.

3.21 Модель на основе дерева решений

После накопления достаточной истории выполнения заданий экспертную систему можно дополнить моделью на основе дерева решений. Такая модель использует обучающую выборку из прошлых квестов. В качестве входных признаков выступают категория, сложность, награда, срок выполнения, день недели, время суток, число активных заданий, количество напоминаний, число отклонений и история успешности похожих поручений.

Целевой переменной может быть успешность задания. Значение 1 присваивается квесту, если он был выполнен, одобрен и доведен до получения награды без значительной задержки. Значение 0 присваивается заданию, если оно было отменено, просрочено, часто отклонялось или долго не переходило к отправке на проверку.

Преимущество дерева решений заключается в объяснимости. В отличие от более сложных моделей, дерево можно представить в виде последовательности условий: если категория – чтение, среднее время выполнения меньше суток, а число отказов низкое, то вероятность успешного выполнения высокая. Такая логика понятна родителю и хорошо сочетается с экспертной системой.

На этапе MVP дерево решений может быть описано как направление развития, а экспертная система используется как первый работающий механизм. В дальнейшем оба подхода могут работать совместно: дерево решений оценивает вероятность успешного выполнения, а экспертная система добавляет педагогические ограничения и объяснение.

3.22 Гибридный подход к рекомендациям

Гибридный подход позволяет объединить преимущества экспертных правил и дерева решений. Экспертная система хорошо работает при малом объеме данных и дает управляемые объяснения. Дерево решений лучше учитывает реальные закономерности, когда в системе накопилась история выполнения квестов.

Итоговая оценка может рассчитываться как комбинация двух частей: expertScore и modelScore. ExpertScore формируется правилами, а modelScore показывает вероятность успешного выполнения, полученную моделью. Например, итоговая оценка может рассчитываться как 0.6 * expertScore + 0.4 * modelScore. На раннем этапе, пока данных мало, вес экспертной системы должен быть выше. После накопления истории вес дерева решений можно постепенно увеличивать.

Такой подход снижает риск ошибочных рекомендаций. Если модель по статистике предлагает слишком сложное задание, экспертные правила могут понизить итоговую оценку из-за перегрузки ребенка или высокой частоты отказов. Если же экспертные правила нейтральны, но история показывает устойчивую успешность категории, дерево решений может повысить рекомендацию.

3.23 Объяснимость рекомендаций для родителя

Объяснимость является обязательным требованием для рекомендательной системы в семейном приложении. Родитель должен понимать, почему система предлагает определенный квест. Простая фраза «рекомендуется» недостаточна, потому что она не раскрывает основание вывода и может восприниматься как случайная подсказка.

Поэтому каждая рекомендация сопровождается текстовыми причинами. Они формируются из сработавших правил экспертной системы. Например: "категория чтения имеет высокий процент завершения", "задания

выполняются в среднем за 18 часов", "за последние семь дней не было отклонений". Если рекомендация отрицательная, система показывает риски: "в категории выросло число отказов", "требуется много напоминаний", "на день уже назначено пять активных заданий".

Такое объяснение делает модуль не только технической функцией, но и инструментом диалога. Родитель может согласиться с рекомендацией, изменить параметры задания или сознательно проигнорировать совет. Система не навязывает решение, а показывает основание для выбора.

3.24 Интеграция экспертного модуля в Firebase и веб-панель

Экспертный модуль интегрирован между аналитическими витринами Firestore и формой создания квеста. Клиентский recommendationEngine применяет типизированный набор правил и возвращает результат непосредственно интерфейсу.

Результаты пересчета сохраняются в подколлекциях analyticsDaily, analyticsByCategory и recommendations. Правила Firestore разрешают запись агрегатов только аутентифицированному родителю той же семьи. Для производственной конфигурации подготовлены эквивалентные callable-функции, однако их развертывание требует тарифа Firebase Blaze. [11]

Unity-клиент отправляет события через AnalyticsEventService и продолжает выполнять основную игровую роль: показывает задания, передает выполнение на проверку и отображает прогресс. Родительская веб-панель отвечает за анализ и принятие решения.

3.25 Этапы разработки аналитико-рекомендательного модуля

Разработка выполнена поэтапно. Сначала добавлена фиксация событий жизненного цикла квеста в Unity и Web. Затем реализованы агрегаты по дням и категориям и экран аналитики родительской панели.

На следующем этапе создана база экспертных правил, механизм расчета оценки и формат объяснимого результата. Рекомендации

интегрированы в форму создания квеста вместе с готовыми шаблонами и автоматическим расчетом награды по сложности.

Дальнейшим этапом остается обучение дерева решений после накопления достаточной и репрезентативной истории. До этого момента правила экспертной системы обеспечивают предсказуемое поведение и прозрачное объяснение.

3.26 Технические риски и способы их снижения

Главный риск аналитического модуля связан с качеством данных. Если события будут записываться неполно или дублироваться, рекомендации станут неточными. Для снижения риска необходимо централизовать отправку событий через AnalyticsEventService в Unity и analyticsService в веб-приложении. Для события quest_viewed следует использовать защиту от повторной записи при каждом обновлении интерфейса.

Второй риск связан с преждевременным использованием сложной модели. Если данных мало, дерево решений может давать случайные выводы. Поэтому первым этапом выбрана экспертная система: она работает на понятных правилах и не требует большой обучающей выборки. Дерево решений должно подключаться только после накопления достаточной истории.

Третий риск связан с приватностью. Система анализирует поведение ребенка, поэтому нельзя превращать ее в инструмент скрытого контроля. Нужно хранить только те данные, которые нужны для работы приложения: события квестов, статусы, временные метки, категории и агрегированные показатели. Для геолокационных квестов следует фиксировать факт посещения, а не постоянный маршрут ребенка.

Четвертый риск связан с безопасностью. Ребенок не должен иметь возможность вручную изменять аналитику, рекомендации или начислять себе награды. Поэтому агрегаты, рекомендации и операции начисления

наград в итоговой версии должны рассчитываться на стороне Cloud Functions или другого доверенного backend-слоя.

3.27 Результаты реализации и направления развития

Реализованный аналитико-рекомендательный контур расширил родительское приложение: помимо журнала заданий оно показывает динамику выполнения и формирует объяснимые советы для создания новых квестов.

Экспертная система работает при небольшом объеме данных, поскольку использует детерминированные правила. По мере накопления истории ее можно дополнить деревом решений, сохранив экспертные ограничения и текстовые объяснения.

Геолокационные задания остаются направлением дальнейшего развития. Их внедрение потребует отдельного Unity-сервиса или Android-плагина, явного управления разрешениями и минимизации сохраняемых координат. Такое разделение позволяет не выдавать проектную модель геолокации за завершенную функцию.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы исходный Unity-прототип геймифицированного ежедневника преобразован в распределенную программную систему, объединяющую игровой клиент ребенка, родительскую веб-панель и облачное хранилище данных.

В Unity доработаны авторизация, адаптивный интерфейс, карта с объектными локациями, доска приключений, полноэкранный просмотр задания и визуальное начисление опыта и золота. Графические ресурсы организованы в единую библиотеку и используются при построении игровых экранов.

Веб-приложение обеспечивает создание семьи и профиля ребенка, выдачу данных для входа, создание и проверку квестов, использование готовых шаблонов и автоматический расчет награды по сложности. Синхронизация Unity и Web реализована через Firebase Authentication и Cloud Firestore с разграничением доступа правилами безопасности.

Дополнительно реализован аналитико-рекомендательный модуль. Он фиксирует события жизненного цикла заданий, строит агрегаты по дням и категориям и применяет объяснимые экспертные правила. Родитель получает оценку рекомендации, причины, риски и предлагаемые параметры нового квеста.

Проверка проекта включала сборку Next.js-приложения, компиляцию функций и Unity-скриптов, а также прохождение основных пользовательских сценариев. Выявленное ограничение бесплатного тарифа Firebase учтено: аналитика имеет защищенный клиентский режим пересчета, а серверные Cloud Functions подготовлены для последующего развертывания.

Поставленная цель достигнута. Разработанная система связывает реальные обязанности ребенка с игровым прогрессом, предоставляет родителю инструменты управления и анализа и создает основу для

дальнейшего внедрения геолокационных квестов и обучаемой рекомендательной модели.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Deterding, S. From game design elements to gamefulness: defining gamification / S. Deterding, D. Dixon, R. Khaled, L. Nacke // Proceedings of the 15th International Academic MindTrek Conference. – New York : ACM, 2011. – P. 9–15. – DOI 10.1145/2181037.2181040.

2. Kapp, K. M. The Gamification of Learning and Instruction: Game-based Methods and Strategies for Training and Education / K. M. Kapp. – San Francisco : Pfeiffer, 2012. – 336 p. – ISBN 978-1-118-09634-5.

3. Werbach, K. For the Win: How Game Thinking Can Revolutionize Your Business / K. Werbach, D. Hunter. – Philadelphia : Wharton Digital Press, 2012. – 148 p. – ISBN 978-1-61363-023-5.

4. Шелл, Дж. Геймдизайн. Как создать игру, в которую будут играть все / Дж. Шелл. – Москва : Альпина Паблишер, 2019. – 640 с. – ISBN 978-5-9614-2512-3.

5. Unity 6.4 User Manual // Unity Documentation : [сайт]. – URL: https://docs.unity3d.com/6000.4/Documentation/Manual/ (дата обращения: 12.06.2026).

6. Designing UI for Multiple Resolutions // Unity Documentation : [сайт]. – URL: https://docs.unity3d.com/Manual/HOWTO-UIMultiResolution.html (дата обращения: 12.06.2026).

7. Input System // Unity Documentation : [сайт]. – URL: https://docs.unity3d.com/Packages/com.unity.inputsystem@1.14/manual/ (дата обращения: 12.06.2026).

8. Асинхронное программирование с использованием async и await // Microsoft Learn : [сайт]. – URL: https://learn.microsoft.com/dotnet/csharp/asynchronous-programming/ (дата обращения: 12.06.2026).

9. Firebase Authentication // Firebase : [сайт]. – URL: https://firebase.google.com/docs/auth (дата обращения: 12.06.2026).

10. Cloud Firestore Data Model // Firebase : [сайт]. – URL: https://firebase.google.com/docs/firestore/data-model (дата обращения: 12.06.2026).

11. Cloud Firestore Security Rules // Firebase : [сайт]. – URL: https://firebase.google.com/docs/firestore/security/ (дата обращения: 12.06.2026).

12. Callable Functions // Firebase : [сайт]. – URL: https://firebase.google.com/docs/functions/callable (дата обращения: 12.06.2026).

13. App Router // Next.js : [сайт]. – URL: https://nextjs.org/docs/app (дата обращения: 12.06.2026).

14. React // [сайт]. – URL: https://react.dev/learn (дата обращения: 12.06.2026).

15. TypeScript Handbook // TypeScript : [сайт]. – URL: https://www.typescriptlang.org/docs/handbook/ (дата обращения: 12.06.2026).

16. Norman, D. A. The Design of Everyday Things / D. A. Norman. – Revised and expanded ed. – New York : Basic Books, 2013. – 368 p. – ISBN 978-0-465-05065-9.

17. LocationService.Start // Unity : [сайт]. – URL: https://docs.unity3d.com/6000.4/Documentation/ScriptReference/LocationService.Start.html (дата обращения: 12.06.2026).

18. Geofencing // Android Developers : [сайт]. – URL: https://developer.android.com/develop/sensors-and-location/location/geofencing (дата обращения: 12.06.2026).

19. Location Permissions // Android Developers : [сайт]. – URL: https://developer.android.com/develop/sensors-and-location/location/permissions (дата обращения: 12.06.2026).

20. Decision Trees // scikit-learn : [сайт]. – URL: https://scikit-learn.org/stable/modules/tree.html (дата обращения: 12.06.2026).